GOSIM

GOSIM Shenzhen 2026

FROM PREDICTION TO CONSEQUENCE

A NEW EVALUATION PARADIGM FOR EMBODIED AI.

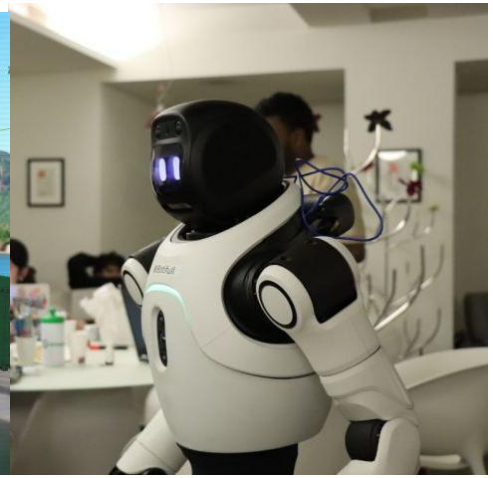
从预测到实际后果

具身智能评测的新范式

Alba María **Téllez Fernández** · 田爱丽
Independent Researcher





Alba María Téllez Fernández

Independent Researcher
AI Lead at Marso Robotics

amtellezfernandez@gmail.com

Can I actually compare these models?

Different architectures produce different objects — so what would a fair leaderboard measure?

VLA

Action chunks

↓

Typical target: task success

WORLD MODEL

Predicted / latent futures

↓

Typical target: prediction / planning

HIERARCHICAL AGENT

Skills and decisions

↓

Typical target: long-horizon success

What would a fair leaderboard even measure?

They have different outputs, action interfaces, observation & replanning schedules and definitions of success.

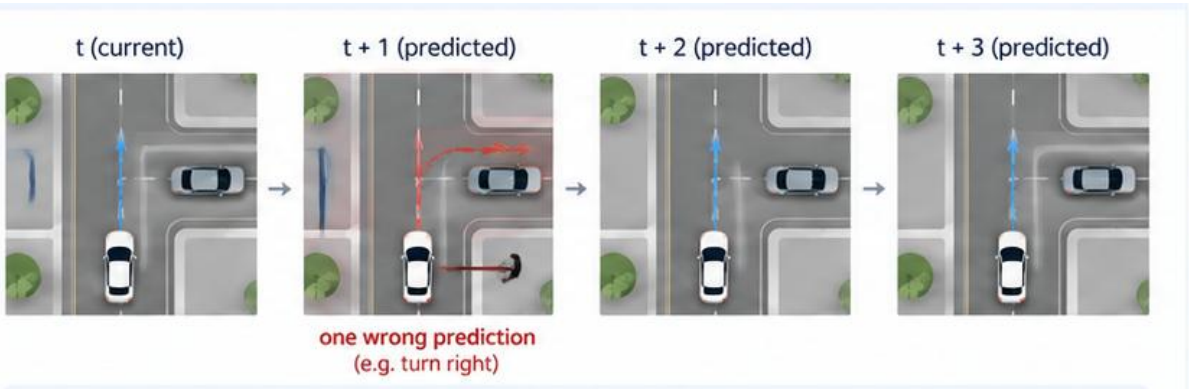
Before comparing models, compare what we measure.

Open-loop measures predictions against fixed references; closed-loop measures behavior after actions change the world.

OPEN LOOP (Prediction scores)

OBSERVE → PREDICT → COMPARE

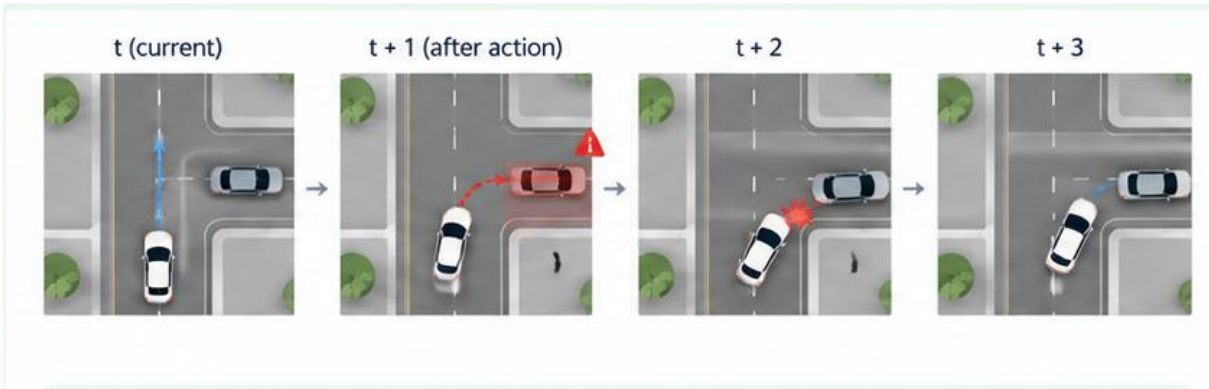
Compare against recorded references



CLOSED LOOP (Executed output)

OBSERVE → PREDICT → EXECUTE

↶ NEW STATE → NEW OBSERVATION



Similar prediction errors. Different physical outcomes.

Illustration: equal local action-error magnitude can lead to recovery or failure depending on the physical state.

Recoverable

Similar prediction error, task can still succeed.



1. Approach (prediction)



2. Slightly misaligned



3. Correct and grasp

Task success

The object remains reachable and the robot can recover from the error.

Critical

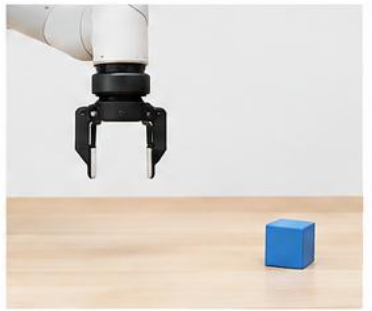
Similar prediction error, task fails due to physical consequences.



1. Approach (prediction)



2. Contact with error



3. Object out of reach

Task failure

The object is moved out of reach, and the task cannot be completed.

Prediction accuracy ≠ task success.

Open-loop tests predictions; closed-loop tests outcomes. But two policies with the same task-success rate may behave very differently.

	OPEN LOOP	CLOSED LOOP
QUESTION	Did the model predict correctly?	Did the robot complete the task?
CAN MEASURE	**Prediction errors, sensitivity, latent quality**	**Success; recovery and safety when tested; execution cost**
EXAMPLES	Action MSE on DROID · video FVD on BAIR	LIBERO · CALVIN · RoboCasa
LIMITATION	No executed downstream consequences	A success rate alone omits replanning, compute, recovery and unscored events

Prediction accuracy ≠ task success.

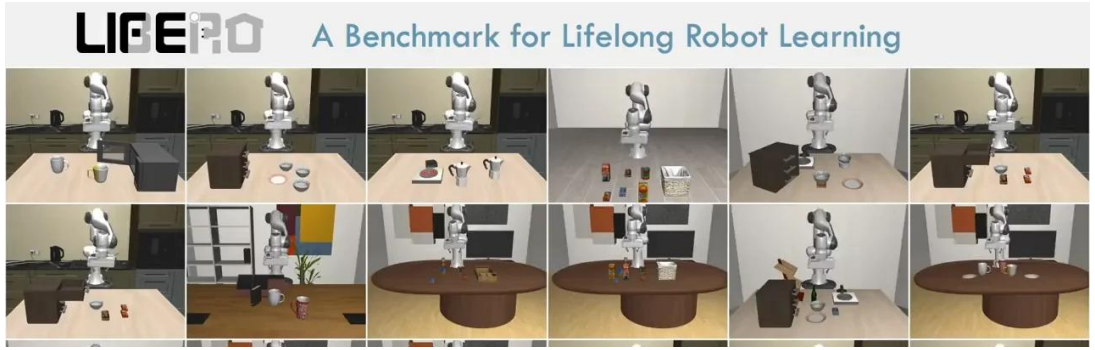
Open-loop tests predictions; closed-loop tests outcomes. But two policies with the same task-success rate may behave very differently.

	OPEN LOOP	CLOSED LOOP
QUESTION	Did the model predict correctly?	Did the robot complete the task?
CAN MEASURE	**Prediction errors, sensitivity, latent quality**	**Success; recovery and safety when tested; execution cost**
EXAMPLES	Action MSE on DROID · video FVD on BAIR	LIBERO · CALVIN · RoboCasa
LIMITATION	No executed downstream consequences	A success rate alone omits replanning, compute, recovery and unscored events

Beyond task success: How did it succeed, at what cost, and what happened during failures?

Preprint
NeurIPS Workshop 2026
RoboPAD

Experimental setup
Frozen SmolVLA policy ·
LIBERO closed-loop evaluation



Refinement versus Refresh: Compute Allocation for Closed-Loop Robot Policies

Anonymous Author(s)
Affiliation
Address
email

Abstract

Robot policies increasingly spend substantial inference compute refining actions before execution, but it is unclear whether this extra computation improves closed-loop control. We report an empirical study in progress of this deployment trade-off in one frozen SmolVLA policy, combining a paired, 12-configuration discovery study with a prespecified comparison on eight LIBERO tasks excluded from configuration selection. At a fixed two-action prefix, reducing denoising from five steps to two increases success from 122/160 to 139/160: a paired difference of 10.6 percentage points with a task-conditional, descriptive 95% interval of 4.4–16.9 points. The observed direction agrees with the frozen prediction of 4.0 points, but its magnitude is larger. Relative to the same comparator, reference-calibrated dense arithmetic is 5.9% lower per control step; mean arithmetic per attempt is 41.1 versus 48.7 TFLOPs, including failures. The 480-attempt discovery study varies both refinement and execution prefix, and full-generation calibration assigns 82.1% of native counted arithmetic to denoising-independent work. A simple cost-aware rule matches the fitted response model's configuration choices at three budget ceilings. For this checkpoint, the separate-task comparison shows that reducing refinement can improve closed-loop success while lowering computation. These findings motivate further study of when additional refinement improves quality and when computation is better spent on more frequent observation-conditioned predictions.

Gap 1 — Compute utility: how the model ran?

The same model weights can produce different success rates depending on inference configuration.



Does more inference actually improve control?

More denoising, lower success.

2 vs 5 steps: +10.6 pp success (95% paired CI: +4.4 to +16.9) · 5.9% less arithmetic

Gap 2 — Compute efficiency: what it cost?

Two controllers can complete a similar number of tasks while one uses over 5× more inference computation.

ACTIONS EXECUTED BEFORE REPLANNING

MEASURE	h = 1	h = 2	h = 5
Success	137/160	139/160	138/160
TFLOPs / attempt	83.3	41.1	16.1

What does achieving success cost?

Similar success, 5.2× difference in computation.

Gap 3 — Feedback freshness: how fresh its inputs were?

Replanning frequently does not guarantee decisions use fresh sensor data.

CURRENT SENSOR DATA

28 / 40

70% success

5-TICK-OLD SENSOR DATA

20 / 40

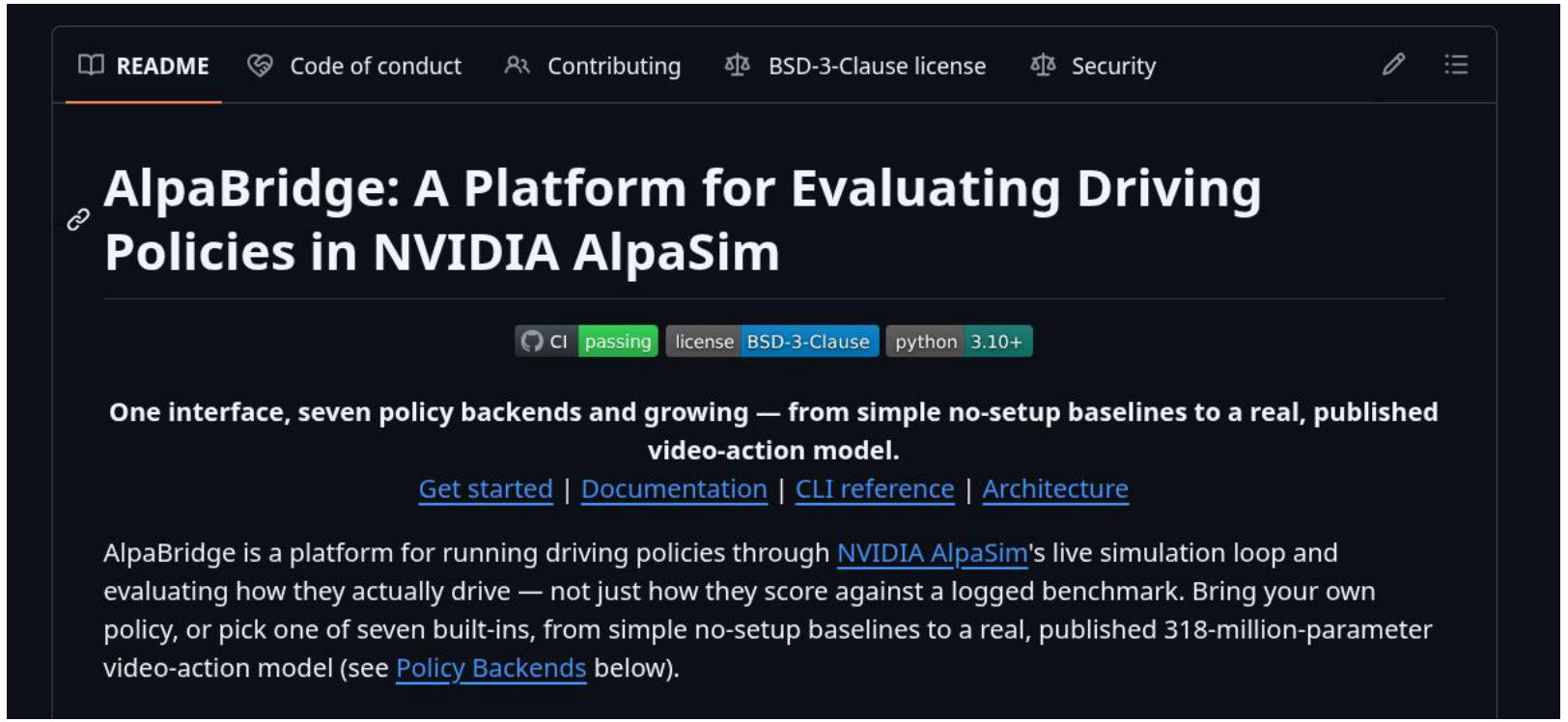
50% success

Is the controller acting on current information?

Same replanning rate, 20 pp success drop with stale inputs.

From task scores to reproducible evaluation

Hold scenarios, interfaces and scoring explicit; AlpaBridge is one driving reference.



- 7 driving-policy backends
- Common policy interface
- Run records + evidence

Best Simulation Environment Prize

AV Commission · SoTA · Apr 2026

Change the model or the evaluation—not the interface.

A shared interface connects different driving policies to different closed-loop evaluations.

MODEL

Driving policies

Baseline · Planner
Learned policy

→

COMMON INTERFACE

AlpaBridge

Observation in
Trajectory out · Logs

→

TEST

Evaluation setups

Scenarios · Simulation
Scoring rules

ONE CLOSED-LOOP CYCLE

Observe → Predict → Execute → New state → Score

What happens at each simulation step?

The simulator supplies observations; the policy returns a trajectory to execute.

One AlpaSim Drive() request, in and out: camera + state in, trajectory_xy out

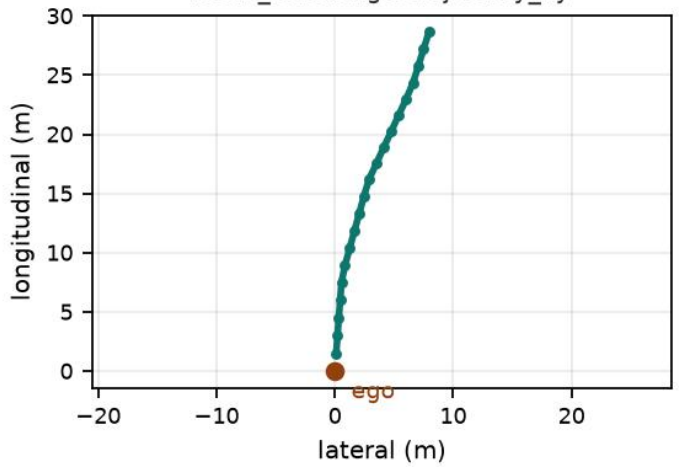
camera_images["camera_front_wide_120fov"]



what AlpaBridge reads from AlpaSim

```
command          = DriveCommand.RIGHT
speed            = 6.0 m/s
acceleration     = 0.2 m/s^2
route_waypoints  = 6 points (x, y)
  first: (0.0, 0.0)
  last:  (40.0, 9.6)
ego_pose_history = [<latest pose>]
scene_id         = "readme-example-scene"
```

route_following's trajectory_xy



The benchmark must preserve the exact observation, output trajectory, and executed consequence.

Two real rollouts. Two very different observations.

The map shows the executed path; the camera panel shows the observations supplied during driving.



TOP: LIVE CAMERA

Camera changes as the vehicle follows its planned route.

BOTTOM: FROZEN INPUT

A camera-blind NAVSIM policy receives an unchanging image while its driven path diverges.

A running simulator is not a valid benchmark by itself.

A score depends on what the policy saw—and which events were allowed to count.

01 · OBSERVATION SOURCE

Matched pose, speed, prompt and history

0.72

Recorded image

0.18

Local rendering

Normalized planned-speed suppression

One local probe at 8 m/s · not a population estimate

02 · SCORING WINDOW

Scoring ends when the declared cutoff is reached

9.9 s

Rear event + cutoff

10.4 s

Later front impact



COUNTED

OUTSIDE WINDOW

Local case studies. Audit camera source, event timestamps, scoring cutoffs and evaluator version.

What makes a closed-loop result reproducible?

Four linked artifacts: test configuration, rollout, execution trace and scorecard.

01

DEFINE

TEST CONFIGURATION

Scene
s · seeds · permitted inputs
Declared scoring rules

→

02

EXECUTE

EXECUTED ROLLOUT

Policy calls · timing
Controller and physics

→

03

TRACE

EXECUTION TRACE

Observations · actions
Physical event timestamps

→

04

SCORE

SCORING RECORD

Event windows · metrics
Uncertainty

ATTACH TO EVERY SCORE **checkpoint · scenario · timing · cutoff rules · uncertainty**

One evidence standard. Different model tests.

Test each model's claimed capability; preserve a common audit of executed behavior.

MODEL TYPE	WHAT TO EVALUATE	REQUIRED EVIDENCE
ACTION POLICY	**Does it complete tasks reliably and efficiently?**	Task success + compute / latency
WORLD MODEL	**Do predicted futures improve decisions?**	Matched execution of alternative actions
HIERARCHICAL AGENT	**Can it recover when a plan fails?**	Perturbed rollouts + recovery trace

Shared audit: inputs · configurations · executed actions · scored events · uncertainty

GOSIM

GOSIM Shenzhen 2026

amtellezfernandez@gmail.com



谢谢!